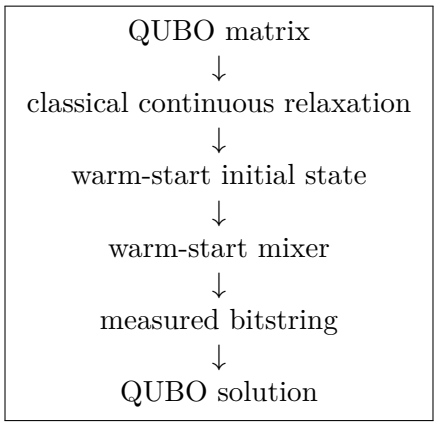
QUBO to warm-start QAOA using Qiskit

By: Sabah Ud Din Ahmad

Last Updated: May 30, 2026

In this document, I explain how to convert a QUBO optimization problem into a warm-start QAOA workflow using Qiskit.

The intended path is:



1. Standard QAOA review

A QUBO problem has the form

$$\boxed{\min_{\mathbf{x}\in\{0,1\}^n} E_{\mathrm{QUBO}}(\mathbf{x}) = \mathbf{x}^{\mathsf{T}} Q \mathbf{x}}$$

where

- $\mathbf{x}$ is a binary decision vector,
- $x_i \in \{0, 1\}$,
- $Q \in \mathbb{R}^{n\times n}$ is the QUBO matrix,
- $E_{\mathrm{QUBO}}(\mathbf{x})$ is the objective value.

The goal is to find

$$\boxed{\mathbf{x}^\star = \arg\min_{\mathbf{x}\in\{0,1\}^n} \mathbf{x}^{\mathsf{T}} Q \mathbf{x}.}$$

Standard QAOA prepares the parameterized state

$$\boxed{|\psi(\gamma,\beta)\rangle = \prod_{\ell=1}^{p} e^{-i\beta_\ell \hat{H}_M} e^{-i\gamma_\ell \hat{H}_C} |+\rangle^{\otimes n}}$$

where

- p is the QAOA depth,
- $\hat{H}_C$ is the cost Hamiltonian encoding the QUBO,
- $\hat{H}_M$ is the mixer Hamiltonian,
- γ and β are variational parameters.

The standard initial state is

$$\boxed{|+\rangle^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)^{\otimes n}.}$$

The standard mixer is usually

$$\boxed{\hat{H}_M = \sum_{i=0}^{n-1} \hat{X}_i.}$$

Standard QAOA therefore starts from an equal superposition over all bitstrings.

2. Warm-start QAOA

2.1 Warm-start initial state

Warm-start QAOA does not start from the uniform superposition. Instead, it uses a **classical relaxation** of the QUBO to bias the initial quantum state.

Original binary QUBO:

$$\boxed{\min_{\mathbf{x}\in\{0,1\}^n} \mathbf{x}^\mathsf{T} Q \mathbf{x}}$$

Continuous relaxation:

$$\boxed{\min_{\mathbf{c}\in[0,1]^n} \mathbf{c}^\mathsf{T} Q \mathbf{c}.}$$

The relaxation replaces

$$x_i \in \{0, 1\}$$

with

$$c_i \in [0, 1].$$

Solving the relaxed problem gives

$$\boxed{\mathbf{c}^\star = (c_0^\star, c_1^\star, \dots, c_{n-1}^\star).}$$

Each relaxed value $c_i^\star$ is then used to initialize qubit i as

$$\boxed{|\psi_i^{\mathrm{WS}}\rangle = \sqrt{1 - c_i^\star}|0\rangle + \sqrt{c_i^\star}|1\rangle.}$$

The full warm-start initial state is

$$\boxed{|\psi_0^{\mathrm{WS}}\rangle = \bigotimes_{i=0}^{n-1} \left(\sqrt{1-c_i^\star}|0\rangle + \sqrt{c_i^\star}|1\rangle\right).}$$

Interpretation:

- If $c_i^\star \approx 0$, qubit i starts close to $|0\rangle$.
- If $c_i^\star \approx 1$, qubit i starts close to $|1\rangle$.
- If $c_i^\star \approx 1/2$, qubit i starts close to $|+\rangle$.

  Standard QAOA starts blindly from the uniform superposition. Warm-start QAOA starts near a classical relaxed solution and lets QAOA refine it.

A single-qubit R_y rotation satisfies

$$R_y(\theta_i)|0\rangle = \cos\left(\frac{\theta_i}{2}\right)|0\rangle + \sin\left(\frac{\theta_i}{2}\right)|1\rangle.$$

We want

$$\cos\left(\frac{\theta_i}{2}\right) = \sqrt{1-c_i^\star}$$

and

$$\sin\left(\frac{\theta_i}{2}\right) = \sqrt{c_i^\star}.$$

Therefore,

$$\boxed{\theta_i = 2\arcsin\sqrt{c_i^\star}.}$$

The warm-start initial state can be prepared by applying

$$\boxed{R_y(\theta_i)}$$

to each qubit.

2.2 Warm-start mixer

A proper warm-start QAOA does not only change the initial state. It also changes the mixer so that the warm-start state is naturally preserved by the mixing structure.

For qubit i, the warm-start mixer Hamiltonian is

$$\boxed{H_{M,i}^{\mathrm{WS}} = \begin{bmatrix} 2c_i^\star - 1 & -2\sqrt{c_i^\star(1-c_i^\star)} \\ -2\sqrt{c_i^\star(1-c_i^\star)} & 1-2c_i^\star \end{bmatrix}.}$$

Equivalently, the corresponding mixer circuit can be implemented as

$$\boxed{R_y(-\theta_i)\ R_z(-2\beta)\ R_y(\theta_i).}$$

Thus, the warm-start QAOA layer uses 'warm-start mixer' instead of 'standard X mixer'.

The warm-start QAOA state is

$$\boxed{|\psi_{\mathrm{WS}}(\gamma,\beta)\rangle = \prod_{\ell=1}^{p} U_M^{\mathrm{WS}}(\beta_\ell) U_C(\gamma_\ell) |\psi_0^{\mathrm{WS}}\rangle.}$$

3. Code

```
import itertools
import time
import numpy as np
import pandas as pd

from scipy.optimize import minimize

from qiskit import QuantumCircuit
from qiskit.circuit import Parameter

from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA

from qiskit.primitives import StatevectorSampler

from qiskit_optimization import QuadraticProgram
from qiskit_optimization.algorithms import MinimumEigenOptimizer
```

3.1 Validate a QUBO matrix

```
def validate_qubo_matrix(Q: np.ndarray) -> np.ndarray:
    """
    Validate and return a QUBO matrix as a NumPy float array.
    """
    Q = np.asarray(Q, dtype=float)

    if Q.ndim != 2:
        raise ValueError("Q must be a two-dimensional matrix.")

    if Q.shape[0] != Q.shape[1]:
        raise ValueError("Q must be square.")

    if not np.all(np.isfinite(Q)):
        raise ValueError("Q must contain only finite real numbers.")

    return Q
```

3.2 QUBO energy

```
def qubo_energy(Q: np.ndarray, x: np.ndarray) -> float:
    """
    Compute the QUBO energy x^T Q x.
```

```
    """
    Q = validate_qubo_matrix(Q)
    x = np.asarray(x, dtype=int)

    return float(x @ Q @ x)
```

3.3 Convert QUBO matrix to Qiskit QuadraticProgram

```
def qubo_matrix_to_quadratic_program(
    Q: np.ndarray,
    name: str = "qubo_problem",
) -> QuadraticProgram:
    """
    Convert a QUBO matrix into a Qiskit QuadraticProgram.

    The objective is:
        minimize x^T Q x

    For non-symmetric Q, the effective off-diagonal coefficient is
    Q[i, j] + Q[j, i] for i < j.
    """
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    qp = QuadraticProgram(name)

    for i in range(n):
        qp.binary_var(name=f"x_{i}")

    linear = {}
    quadratic = {}

    for i in range(n):
        if abs(Q[i, i]) > 1e-12:
            linear[f"x_{i}"] = float(Q[i, i])

    for i in range(n):
        for j in range(i + 1, n):
            coeff = float(Q[i, j] + Q[j, i])
            if abs(coeff) > 1e-12:
                quadratic[(f"x_{i}", f"x_{j}")] = coeff

    qp.minimize(linear=linear, quadratic=quadratic)

    return qp
```

3.4 Brute-force exact QUBO solver

Use brute force only for small n.

```
def brute_force_qubo(Q: np.ndarray):
    """
```

```
    Solve a QUBO exactly by brute force.
    """
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    best_x = None
    best_energy = np.inf
    all_results = []

    for bits in itertools.product([0, 1], repeat=n):
        x = np.array(bits, dtype=int)
        energy = qubo_energy(Q, x)

        all_results.append((x, energy))

        if energy < best_energy:
            best_energy = energy
            best_x = x.copy()

    return best_x, best_energy, all_results
```

3.5 Classical relaxation for warm start

The relaxed QUBO is

$$\boxed{\min_{\mathbf{c}\in[0,1]^n} \mathbf{c}^{\mathsf{T}} Q \mathbf{c}.}$$

If Q is positive semidefinite, this is a convex quadratic program. If Q is indefinite, the relaxation is non-convex, and the solver may find only a local minimum. For that reason, the implementation below uses multistart L-BFGS-B.

```
def solve_relaxed_qubo(
    Q: np.ndarray,
    n_starts: int = 20,
    seed: int = 123,
):
    """
    Solve the continuous relaxation of a QUBO.
    """
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    Qsym = 0.5 * (Q + Q.T)

    def objective(c):
        return float(c @ Q @ c)

    def gradient(c):
        return 2.0 * Qsym @ c

    bounds = [(0.0, 1.0) for _ in range(n)]
```

```
    rng = np.random.default_rng(seed)

    initial_points = [np.full(n, 0.5)]
    initial_points += [rng.random(n) for _ in range(max(0, n_starts - 1))]

    best_result = None

    for c0 in initial_points:
        result = minimize(
            objective,
            c0,
            jac=gradient,
            bounds=bounds,
            method="L-BFGS-B",
        )

        if best_result is None or result.fun < best_result.fun:
            best_result = result

    c_best = np.clip(best_result.x, 0.0, 1.0)
    relaxed_energy = float(c_best @ Q @ c_best)

    return c_best, relaxed_energy, best_result
```

3.6 Build the warm-start initial state

```
def warm_start_angles(
    c: np.ndarray,
    epsilon: float = 1e-3,
) -> np.ndarray:
    """
    Convert relaxed variables c_i into warm-start angles.

    theta_i = 2 arcsin sqrt(c_i)

    epsilon clips values away from exactly 0 and 1.
    """
    c = np.asarray(c, dtype=float)

    if c.ndim != 1:
        raise ValueError("c must be a one-dimensional vector.")

    if not np.all(np.isfinite(c)):
        raise ValueError("c must contain only finite values.")

    if not np.all((0.0 <= c) & (c <= 1.0)):
        raise ValueError("All c values must lie in [0, 1].")

    if not (0.0 <= epsilon < 0.5):
        raise ValueError("epsilon must satisfy 0 <= epsilon < 0.5.")
```

```
    c_clipped = np.clip(c, epsilon, 1.0 - epsilon)

    theta = 2.0 * np.arcsin(np.sqrt(c_clipped))

    return theta


def build_warm_start_initial_state(
    c: np.ndarray,
    epsilon: float = 1e-3,
) -> QuantumCircuit:
    """
    Build the warm-start initial-state circuit.
    """
    theta = warm_start_angles(c, epsilon=epsilon)
    n = len(theta)

    initial_state = QuantumCircuit(n, name="warm_start_initial_state")

    for i, theta_i in enumerate(theta):
        initial_state.ry(theta_i, i)

    return initial_state
```

3.7 Build the warm-start mixer

The warm-start mixer circuit is

$$R_y(-\theta_i)R_z(-2\beta)R_y(\theta_i)$$

on each qubit.

```
def build_warm_start_mixer(
    c: np.ndarray,
    epsilon: float = 1e-3,
) -> QuantumCircuit:
    """
    Build the warm-start mixer circuit.
    """
    theta = warm_start_angles(c, epsilon=epsilon)
    n = len(theta)

    beta = Parameter(" ")
    mixer = QuantumCircuit(n, name="warm_start_mixer")

    for i, theta_i in enumerate(theta):
        mixer.ry(-theta_i, i)
        mixer.rz(-2.0 * beta, i)
        mixer.ry(theta_i, i)
```

```
    return mixer
```

3.8 Standard QAOA solver for benchmarking

```
def solve_qubo_with_standard_qaoa(
    Q: np.ndarray,
    reps: int = 1,
    maxiter: int = 200,
    seed: int = 123,
    initial_point=None,
):
    """
    Solve a QUBO using standard QAOA.
    """
    Q = validate_qubo_matrix(Q)
    qp = qubo_matrix_to_quadratic_program(Q)

    sampler = StatevectorSampler(seed=seed)
    optimizer = COBYLA(maxiter=maxiter)

    qaoa = QAOA(
        sampler=sampler,
        optimizer=optimizer,
        reps=reps,
        initial_point=initial_point,
    )

    optimizer_wrapper = MinimumEigenOptimizer(qaoa)

    start = time.perf_counter()
    result = optimizer_wrapper.solve(qp)
    elapsed = time.perf_counter() - start

    x = np.array(result.x, dtype=int)
    energy = qubo_energy(Q, x)

    return {
        "method": "standard_qaoa",
        "solution": x,
        "qubo_energy": energy,
        "qiskit_result": result,
        "elapsed_seconds": elapsed,
        "quadratic_program": qp,
    }
```

3.9 Warm-start QAOA solver

```
def solve_qubo_with_warm_start_qaoa(
    Q: np.ndarray,
    reps: int = 1,
```

```
    maxiter: int = 200,
    seed: int = 123,
    n_starts: int = 20,
    epsilon: float = 1e-3,
    initial_point=None,
):
    """
    Solve a QUBO using warm-start QAOA.

    Steps:
        1. Solve the continuous relaxation.
        2. Build the warm-start initial state.
        3. Build the warm-start mixer.
        4. Run QAOA with the warm-start initial state and mixer.
    """
    Q = validate_qubo_matrix(Q)
    qp = qubo_matrix_to_quadratic_program(Q)

    c_star, relaxed_energy, relaxed_result = solve_relaxed_qubo(
        Q=Q,
        n_starts=n_starts,
        seed=seed,
    )

    initial_state = build_warm_start_initial_state(
        c=c_star,
        epsilon=epsilon,
    )

    mixer = build_warm_start_mixer(
        c=c_star,
        epsilon=epsilon,
    )

    sampler = StatevectorSampler(seed=seed)
    optimizer = COBYLA(maxiter=maxiter)

    qaoa = QAOA(
        sampler=sampler,
        optimizer=optimizer,
        reps=reps,
        initial_state=initial_state,
        mixer=mixer,
        initial_point=initial_point,
    )

    optimizer_wrapper = MinimumEigenOptimizer(qaoa)

    start = time.perf_counter()
    result = optimizer_wrapper.solve(qp)
    elapsed = time.perf_counter() - start
```

```
    x = np.array(result.x, dtype=int)
    energy = qubo_energy(Q, x)

    return {
        "method": "warm_start_qaoa",
        "solution": x,
        "qubo_energy": energy,
        "relaxed_solution": c_star,
        "relaxed_energy": relaxed_energy,
        "relaxed_result": relaxed_result,
        "initial_state": initial_state,
        "mixer": mixer,
        "qiskit_result": result,
        "elapsed_seconds": elapsed,
        "quadratic_program": qp,
    }
```

4. Example

```
Q = np.array([
    [1, -2,  0],
    [0,  1, -2],
    [0,  0,  1],
], dtype=float)

exact_x, exact_energy, all_results = brute_force_qubo(Q)

standard_out = solve_qubo_with_standard_qaoa(
    Q=Q,
    reps=1,
    maxiter=200,
    seed=123,
)

warm_out = solve_qubo_with_warm_start_qaoa(
    Q=Q,
    reps=1,
    maxiter=200,
    seed=123,
    n_starts=20,
    epsilon=1e-3,
)

print("Exact solution:", exact_x)
print("Exact energy:", exact_energy)

print("Standard QAOA solution:", standard_out["solution"])
print("Standard QAOA energy:", standard_out["qubo_energy"])
```

```
print("Relaxed solution c*:", warm_out["relaxed_solution"])
print("Relaxed energy:", warm_out["relaxed_energy"])

print("Warm-start QAOA solution:", warm_out["solution"])
print("Warm-start QAOA energy:", warm_out["qubo_energy"])
```

Because QAOA is approximate and optimizer-dependent, the QAOA result can vary with seed, depth, optimizer, and Qiskit version.

5. Validation

Validation should always be done using the original QUBO objective

$$\boxed{E_{\mathrm{QUBO}}(\mathbf{x}) = \mathbf{x}^{\mathsf{T}} Q \mathbf{x}.}$$

Do not validate only against the Ising Hamiltonian or the internal Qiskit objective.

```
def validate_against_brute_force(Q, solver_output):
    """
    Compare a QAOA solution against brute force.
    """
    exact_x, exact_energy, _ = brute_force_qubo(Q)

    qaoa_x = solver_output["solution"]
    qaoa_energy = qubo_energy(Q, qaoa_x)

    return {
        "method": solver_output["method"],
        "exact_x": exact_x,
        "exact_energy": exact_energy,
        "qaoa_x": qaoa_x,
        "qaoa_energy": qaoa_energy,
        "energy_gap": qaoa_energy - exact_energy,
        "energy_match": np.isclose(qaoa_energy, exact_energy),
    }


standard_validation = validate_against_brute_force(Q, standard_out)
warm_validation = validate_against_brute_force(Q, warm_out)

print("Standard QAOA validation:")
print(standard_validation)

print("Warm-start QAOA validation:")
print(warm_validation)
```

6. Inspecting sampled probabilities

Qiskit optimization results may contain sampled solutions. When available, we can inspect the probability assigned to the optimal bitstring.

```python
def bitstring_from_array(x):
    """
    Convert an array like [1, 0, 1] to the string '101'.
    """
    return "".join(str(int(v)) for v in x)


def extract_sample_table(result, Q=None, top_k: int = 10):
    """
    Extract a table of sampled bitstrings from a Qiskit Optimization result.
    """
    if not hasattr(result, "samples") or result.samples is None:
        return pd.DataFrame()

    rows = []

    for sample in result.samples:
        x = np.array(sample.x, dtype=int)
        row = {
            "bitstring": bitstring_from_array(x),
            "probability": getattr(sample, "probability", np.nan),
            "qiskit_fval": getattr(sample, "fval", np.nan),
        }

        if Q is not None:
            row["qubo_energy"] = qubo_energy(Q, x)

        rows.append(row)

    df = pd.DataFrame(rows)

    if len(df) == 0:
        return df

    sort_cols = []
    ascending = []

    if "qubo_energy" in df.columns:
        sort_cols.append("qubo_energy")
        ascending.append(True)

    if "probability" in df.columns:
        sort_cols.append("probability")
        ascending.append(False)

    if sort_cols:
        df = df.sort_values(sort_cols, ascending=ascending)

    return df.head(top_k).reset_index(drop=True)
```

```
standard_samples = extract_sample_table(
    standard_out["qiskit_result"],
    Q=Q,
    top_k=10,
)

warm_samples = extract_sample_table(
    warm_out["qiskit_result"],
    Q=Q,
    top_k=10,
)

print("Standard QAOA samples:")
print(standard_samples)

print("Warm-start QAOA samples:")
print(warm_samples)
```

7. Benchmarking standard QAOA vs warm-start QAOA

A useful benchmark should compare:

1. Best solution found.
2. Best QUBO energy found.
3. Gap from the exact brute-force optimum.
4. Whether the exact optimum was found.
5. Probability of sampling the optimum, if available.
6. Runtime.
7. Effect of QAOA depth p.
8. Effect of random seed.

7.1 Probability of exact optimum

```
def probability_of_bitstring(result, target_x):
    """
    Return the probability assigned to a target bitstring in a Qiskit result.

    If samples are unavailable, return np.nan.
    """
    if not hasattr(result, "samples") or result.samples is None:
        return np.nan

    target_x = np.array(target_x, dtype=int)

    prob = 0.0

    for sample in result.samples:
        sample_x = np.array(sample.x, dtype=int)
        if np.array_equal(sample_x, target_x):
            prob += float(getattr(sample, "probability", 0.0))
```

```
    return prob
```

7.2 Benchmarking

```
def benchmark_qaoa_methods(
    Q: np.ndarray,
    reps_list=(1, 2),
    seeds=(123, 456, 789),
    maxiter: int = 200,
    n_starts: int = 20,
    epsilon: float = 1e-3,
):
    """
    Benchmark standard QAOA and warm-start QAOA on one QUBO.
    """
    Q = validate_qubo_matrix(Q)

    exact_x, exact_energy, _ = brute_force_qubo(Q)

    rows = []

    for reps in reps_list:
        for seed in seeds:
            standard_out = solve_qubo_with_standard_qaoa(
                Q=Q,
                reps=reps,
                maxiter=maxiter,
                seed=seed,
            )

            warm_out = solve_qubo_with_warm_start_qaoa(
                Q=Q,
                reps=reps,
                maxiter=maxiter,
                seed=seed,
                n_starts=n_starts,
                epsilon=epsilon,
            )

            for out in [standard_out, warm_out]:
                x = out["solution"]
                energy = qubo_energy(Q, x)

                prob_exact = probability_of_bitstring(
                    out["qiskit_result"],
                    exact_x,
                )

                row = {
                    "method": out["method"],
                    "reps": reps,
```

```
                    "seed": seed,
                    "solution": bitstring_from_array(x),
                    "energy": energy,
                    "exact_energy": exact_energy,
                    "energy_gap": energy - exact_energy,
                    "found_exact": np.isclose(energy, exact_energy),
                    "probability_exact": prob_exact,
                    "elapsed_seconds": out["elapsed_seconds"],
                }

                if out["method"] == "warm_start_qaoa":
                    row["relaxed_energy"] = out["relaxed_energy"]
                    row["relaxed_solution"] = np.array2string(
                        out["relaxed_solution"],
                        precision=3,
                    )
                else:
                    row["relaxed_energy"] = np.nan
                    row["relaxed_solution"] = ""

                rows.append(row)

    return pd.DataFrame(rows)

benchmark_df = benchmark_qaoa_methods(
    Q=Q,
    reps_list=(1, 2),
    seeds=(123, 456, 789),
    maxiter=200,
    n_starts=20,
    epsilon=1e-3,
)

print(benchmark_df)
```

Summary:

```
summary = benchmark_df.groupby(["method", "reps"]).agg(
    success_rate=("found_exact", "mean"),
    mean_energy=("energy", "mean"),
    best_energy=("energy", "min"),
    mean_gap=("energy_gap", "mean"),
    mean_probability_exact=("probability_exact", "mean"),
    mean_elapsed_seconds=("elapsed_seconds", "mean"),
).reset_index()

print(summary)
```

8. Qiskit's built-in `WarmStartQAOAOptimizer`

Qiskit Optimization also provides a built-in `WarmStartQAOAOptimizer`.

Conceptually, it performs the same high-level procedure as explained in previous sections.

A schematic usage pattern is:

```
from qiskit_optimization.algorithms import WarmStartQAOAOptimizer

# Requires a compatible pre_solver.
# Some tutorials use CplexOptimizer if CPLEX is installed.

ws_optimizer = WarmStartQAOAOptimizer(
    pre_solver=pre_solver,
    relax_for_pre_solver=True,
    qaoa=qaoa,
    epsilon=0.0,
)

result = ws_optimizer.solve(qp)
```

9. Assumptions and Limitations

We assume:

1. The problem is already in QUBO form.
2. The objective is a minimization problem.
3. QUBO objective is exactly

$$\mathbf{x}^{\mathsf{T}} Q \mathbf{x}.$$

4. Variables are binary:

$$x_i \in \{0, 1\}.$$

5. QUBO is small enough for local simulation.
6. Brute-force validation is used only for small n.
7. The relaxed solution $c^\star$ is used as a heuristic initialization, not as a proof of optimality.
8. The benchmark uses local simulation, not real quantum hardware.

The limitations are:

1. Warm start is not guaranteed to improve QAOA. Warm-start QAOA can help when the relaxed solution is informative. It can hurt when the relaxed solution biases the circuit toward a poor region.
2. The relaxed solution is not the binary solution. A relaxed solution like $c_i^\star = 0.73$ does not mean the final answer is $x_i = 1$ with certainty. It means the initial qubit is biased toward $|1\rangle$.
3. Hardware behavior can differ from simulation. Real hardware introduces finite shots, gate noise, readout error, transpilation overhead, device connectivity constraints, queue time, and calibration drift. Therefore, simulation benchmarks are not hardware benchmarks.

10. Complete code

```
import itertools
import time
import numpy as np
import pandas as pd

from scipy.optimize import minimize

from qiskit import QuantumCircuit
from qiskit.circuit import Parameter

from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA

from qiskit.primitives import StatevectorSampler

from qiskit_optimization import QuadraticProgram
from qiskit_optimization.algorithms import MinimumEigenOptimizer


def validate_qubo_matrix(Q):
    Q = np.asarray(Q, dtype=float)

    if Q.ndim != 2:
        raise ValueError("Q must be a two-dimensional matrix.")

    if Q.shape[0] != Q.shape[1]:
        raise ValueError("Q must be square.")

    if not np.all(np.isfinite(Q)):
        raise ValueError("Q must contain only finite real numbers.")

    return Q


def qubo_energy(Q, x):
    Q = validate_qubo_matrix(Q)
    x = np.asarray(x, dtype=int)
    return float(x @ Q @ x)


def qubo_matrix_to_quadratic_program(Q, name="qubo_problem"):
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    qp = QuadraticProgram(name)

    for i in range(n):
        qp.binary_var(name=f"x_{i}")
```

```
    linear = {}
    quadratic = {}

    for i in range(n):
        if abs(Q[i, i]) > 1e-12:
            linear[f"x_{i}"] = float(Q[i, i])

    for i in range(n):
        for j in range(i + 1, n):
            coeff = float(Q[i, j] + Q[j, i])
            if abs(coeff) > 1e-12:
                quadratic[(f"x_{i}", f"x_{j}")] = coeff

    qp.minimize(linear=linear, quadratic=quadratic)

    return qp


def brute_force_qubo(Q):
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    best_x = None
    best_energy = np.inf
    all_results = []

    for bits in itertools.product([0, 1], repeat=n):
        x = np.array(bits, dtype=int)
        energy = qubo_energy(Q, x)

        all_results.append((x, energy))

        if energy < best_energy:
            best_energy = energy
            best_x = x.copy()

    return best_x, best_energy, all_results


def solve_relaxed_qubo(Q, n_starts=20, seed=123):
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    Qsym = 0.5 * (Q + Q.T)

    def objective(c):
        return float(c @ Q @ c)

    def gradient(c):
        return 2.0 * Qsym @ c
```

```
    bounds = [(0.0, 1.0) for _ in range(n)]

    rng = np.random.default_rng(seed)

    initial_points = [np.full(n, 0.5)]
    initial_points += [rng.random(n) for _ in range(max(0, n_starts - 1))]

    best_result = None

    for c0 in initial_points:
        result = minimize(
            objective,
            c0,
            jac=gradient,
            bounds=bounds,
            method="L-BFGS-B",
        )

        if best_result is None or result.fun < best_result.fun:
            best_result = result

    c_best = np.clip(best_result.x, 0.0, 1.0)
    relaxed_energy = float(c_best @ Q @ c_best)

    return c_best, relaxed_energy, best_result


def warm_start_angles(c, epsilon=1e-3):
    c = np.asarray(c, dtype=float)

    if c.ndim != 1:
        raise ValueError("c must be a one-dimensional vector.")

    if not np.all(np.isfinite(c)):
        raise ValueError("c must contain only finite values.")

    if not np.all((0.0 <= c) & (c <= 1.0)):
        raise ValueError("All c values must lie in [0, 1].")

    if not (0.0 <= epsilon < 0.5):
        raise ValueError("epsilon must satisfy 0 <= epsilon < 0.5.")

    c_clipped = np.clip(c, epsilon, 1.0 - epsilon)

    return 2.0 * np.arcsin(np.sqrt(c_clipped))


def build_warm_start_initial_state(c, epsilon=1e-3):
    theta = warm_start_angles(c, epsilon=epsilon)
    n = len(theta)
```

```
    initial_state = QuantumCircuit(n, name="warm_start_initial_state")

    for i, theta_i in enumerate(theta):
        initial_state.ry(theta_i, i)

    return initial_state


def build_warm_start_mixer(c, epsilon=1e-3):
    theta = warm_start_angles(c, epsilon=epsilon)
    n = len(theta)

    beta = Parameter(" ")
    mixer = QuantumCircuit(n, name="warm_start_mixer")

    for i, theta_i in enumerate(theta):
        mixer.ry(-theta_i, i)
        mixer.rz(-2.0 * beta, i)
        mixer.ry(theta_i, i)

    return mixer


def solve_qubo_with_standard_qaoa(
    Q,
    reps=1,
    maxiter=200,
    seed=123,
    initial_point=None,
):
    Q = validate_qubo_matrix(Q)
    qp = qubo_matrix_to_quadratic_program(Q)

    sampler = StatevectorSampler(seed=seed)
    optimizer = COBYLA(maxiter=maxiter)

    qaoa = QAOA(
        sampler=sampler,
        optimizer=optimizer,
        reps=reps,
        initial_point=initial_point,
    )

    optimizer_wrapper = MinimumEigenOptimizer(qaoa)

    start = time.perf_counter()
    result = optimizer_wrapper.solve(qp)
    elapsed = time.perf_counter() - start

    x = np.array(result.x, dtype=int)
    energy = qubo_energy(Q, x)
```

```
    return {
        "method": "standard_qaoa",
        "solution": x,
        "qubo_energy": energy,
        "qiskit_result": result,
        "elapsed_seconds": elapsed,
        "quadratic_program": qp,
    }


def solve_qubo_with_warm_start_qaoa(
    Q,
    reps=1,
    maxiter=200,
    seed=123,
    n_starts=20,
    epsilon=1e-3,
    initial_point=None,
):
    Q = validate_qubo_matrix(Q)
    qp = qubo_matrix_to_quadratic_program(Q)

    c_star, relaxed_energy, relaxed_result = solve_relaxed_qubo(
        Q=Q,
        n_starts=n_starts,
        seed=seed,
    )

    initial_state = build_warm_start_initial_state(
        c=c_star,
        epsilon=epsilon,
    )

    mixer = build_warm_start_mixer(
        c=c_star,
        epsilon=epsilon,
    )

    sampler = StatevectorSampler(seed=seed)
    optimizer = COBYLA(maxiter=maxiter)

    qaoa = QAOA(
        sampler=sampler,
        optimizer=optimizer,
        reps=reps,
        initial_state=initial_state,
        mixer=mixer,
        initial_point=initial_point,
    )
```

```
    optimizer_wrapper = MinimumEigenOptimizer(qaoa)

    start = time.perf_counter()
    result = optimizer_wrapper.solve(qp)
    elapsed = time.perf_counter() - start

    x = np.array(result.x, dtype=int)
    energy = qubo_energy(Q, x)

    return {
        "method": "warm_start_qaoa",
        "solution": x,
        "qubo_energy": energy,
        "relaxed_solution": c_star,
        "relaxed_energy": relaxed_energy,
        "relaxed_result": relaxed_result,
        "initial_state": initial_state,
        "mixer": mixer,
        "qiskit_result": result,
        "elapsed_seconds": elapsed,
        "quadratic_program": qp,
    }


def bitstring_from_array(x):
    return "".join(str(int(v)) for v in x)


def probability_of_bitstring(result, target_x):
    if not hasattr(result, "samples") or result.samples is None:
        return np.nan

    target_x = np.array(target_x, dtype=int)
    prob = 0.0

    for sample in result.samples:
        sample_x = np.array(sample.x, dtype=int)
        if np.array_equal(sample_x, target_x):
            prob += float(getattr(sample, "probability", 0.0))

    return prob


def benchmark_qaoa_methods(
    Q,
    reps_list=(1, 2),
    seeds=(123, 456, 789),
    maxiter=200,
    n_starts=20,
    epsilon=1e-3,
):
```

```
Q = validate_qubo_matrix(Q)

exact_x, exact_energy, _ = brute_force_qubo(Q)

rows = []

for reps in reps_list:
    for seed in seeds:
        standard_out = solve_qubo_with_standard_qaoa(
            Q=Q,
            reps=reps,
            maxiter=maxiter,
            seed=seed,
        )

        warm_out = solve_qubo_with_warm_start_qaoa(
            Q=Q,
            reps=reps,
            maxiter=maxiter,
            seed=seed,
            n_starts=n_starts,
            epsilon=epsilon,
        )

        for out in [standard_out, warm_out]:
            x = out["solution"]
            energy = qubo_energy(Q, x)

            prob_exact = probability_of_bitstring(
                out["qiskit_result"],
                exact_x,
            )

            row = {
                "method": out["method"],
                "reps": reps,
                "seed": seed,
                "solution": bitstring_from_array(x),
                "energy": energy,
                "exact_energy": exact_energy,
                "energy_gap": energy - exact_energy,
                "found_exact": np.isclose(energy, exact_energy),
                "probability_exact": prob_exact,
                "elapsed_seconds": out["elapsed_seconds"],
            }

            if out["method"] == "warm_start_qaoa":
                row["relaxed_energy"] = out["relaxed_energy"]
                row["relaxed_solution"] = np.array2string(
                    out["relaxed_solution"],
                    precision=3,
```

```
                    )
                else:
                    row["relaxed_energy"] = np.nan
                    row["relaxed_solution"] = ""

                rows.append(row)

    return pd.DataFrame(rows)


if __name__ == "__main__":
    Q = np.array([
        [1, -2,  0],
        [0,  1, -2],
        [0,  0,  1],
    ], dtype=float)

    exact_x, exact_energy, _ = brute_force_qubo(Q)

    standard_out = solve_qubo_with_standard_qaoa(
        Q=Q,
        reps=1,
        maxiter=200,
        seed=123,
    )

    warm_out = solve_qubo_with_warm_start_qaoa(
        Q=Q,
        reps=1,
        maxiter=200,
        seed=123,
        n_starts=20,
        epsilon=1e-3,
    )

    print("Exact solution:", exact_x)
    print("Exact energy:", exact_energy)

    print("Standard QAOA solution:", standard_out["solution"])
    print("Standard QAOA energy:", standard_out["qubo_energy"])

    print("Relaxed solution c*:", warm_out["relaxed_solution"])
    print("Relaxed energy:", warm_out["relaxed_energy"])

    print("Warm-start QAOA solution:", warm_out["solution"])
    print("Warm-start QAOA energy:", warm_out["qubo_energy"])

    benchmark_df = benchmark_qaoa_methods(
        Q=Q,
        reps_list=(1, 2),
        seeds=(123, 456, 789),
```

```
    maxiter=200,
    n_starts=20,
    epsilon=1e-3,
)

print("\nBenchmark:")
print(benchmark_df)

summary = benchmark_df.groupby(["method", "reps"]).agg(
    success_rate=("found_exact", "mean"),
    mean_energy=("energy", "mean"),
    best_energy=("energy", "min"),
    mean_gap=("energy_gap", "mean"),
    mean_probability_exact=("probability_exact", "mean"),
    mean_elapsed_seconds=("elapsed_seconds", "mean"),
).reset_index()

print("\nBenchmark summary:")
print(summary)
```

11. References

1. Daniel J. Egger, Jakub Mareček, and Stefan Woerner, **"Warm-starting quantum optimization,"** *Quantum* 5, 479 (2021).
2. Qiskit Optimization tutorial, "Warm-starting quantum optimization"
3. Qiskit Optimization API documentation, `WarmStartQAOAOptimizer`
4. Qiskit Optimization API documentation, `WarmStartQAOAFactory`
5. Reuben Tate, Majid Farhadi, Creston Herold, Greg Mohler, and Swati Gupta, **"Bridging Classical and Quantum with SDP initialized warm-starts for QAOA,"** *ACM Transactions on Quantum Computing* 4, 2 (2023).
6. Edward Farhi, Jeffrey Goldstone, and Sam Gutmann, **"A Quantum Approximate Optimization Algorithm,"** arXiv:1411.4028 [quant-ph].
7. Qiskit Optimization documentation, `QuadraticProgram`
8. Qiskit Optimization documentation, `MinimumEigenOptimizer`